РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

дисциплина: Архитектура компьютера

Студент: Иванова Анна

Группа: НКАбд-02-25

МОСКВА

2025 г.

Цель работы

Освоение арифметических инструкций языка ассемблера NASM.

Выполнение лабораторной работы

6.3.1. Символьные и численные данные в NASM

Создаю каталог для программам лабораторной работы № 6, перехожу в него и создаю файл lab6-1.asm. (рис 6.1)

```
asivanova@fedora:~$ mkdir ~/work/arch-pc/lab06
asivanova@fedora:~$ cd ~/work/arch-pc/lab06
asivanova@fedora:~/work/arch-pc/lab06$ touch lab6-1.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
lab6-1.asm
```

Рис 6.1 Создание рабочего каталога и файла

Далее копируй файл in_out.asm в текущий каталогб чтобы подпрограммы корректно работали (рис 6.2)

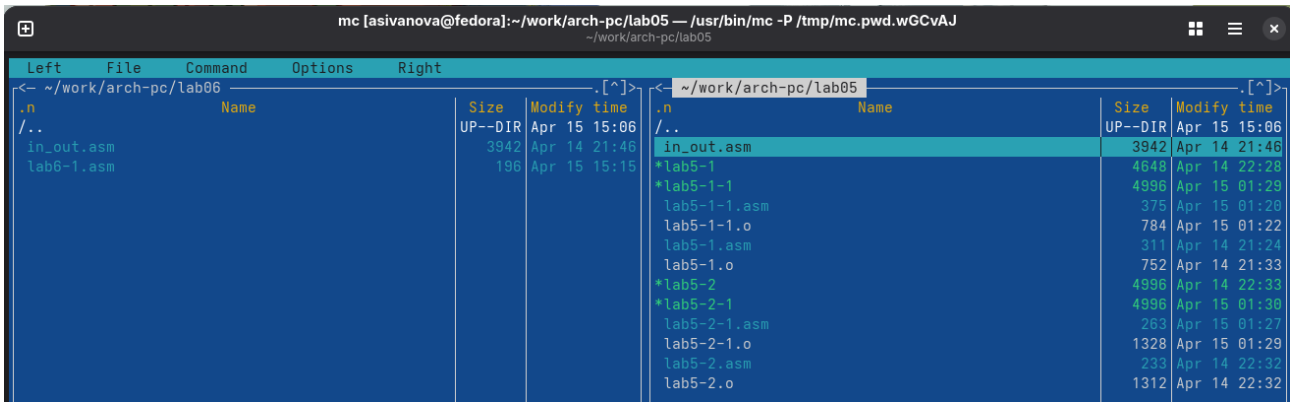
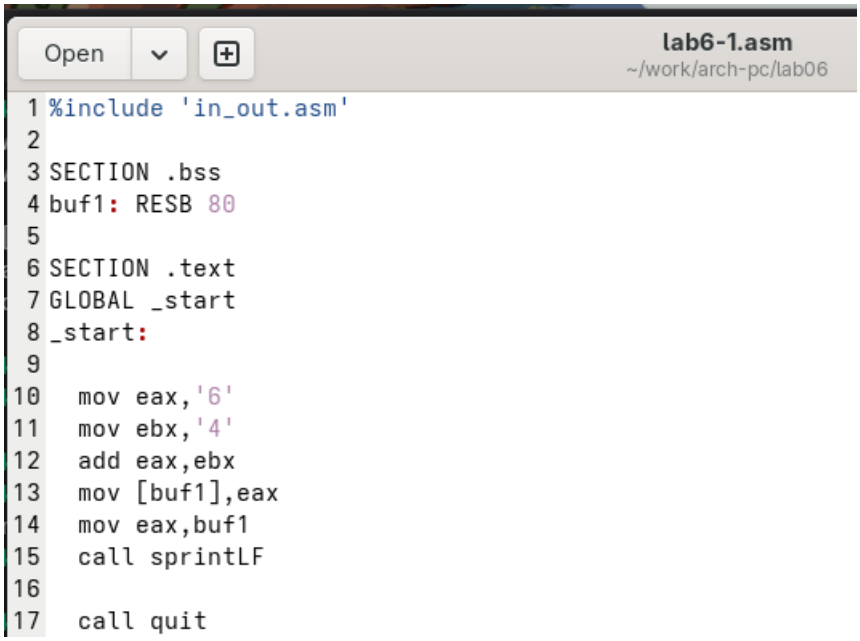


Рис 6.2 Копирование файла in_out.asm

Ввожу в файл lab6-1.asm текст программы из листинга 6.1. (рис 6.3)




```
%include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:

  mov eax,'6'
  mov ebx,'4'
  add eax,ebx
  mov [buf1],eax
  mov eax,buf1
  call sprintLF

  call quit
```

Рис 6.3 Программа из листинга 6.1

Создаю исполняемый файл и проверяю его работу. В результате программа выводит символ j, так как мы складывали коды символов 6 и 4 и получили в итоге код символа j. (рис 6.4)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1.asm  lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-1
j
```

Рис 6.4 Вывод программы

Далее изменяю текст программы и вместо символов записываю в регистры числаб для этого убираю кавычки. (рис 6.5)

Open

lab6-1.asm
~/work/arch-pc/lab06

```
%include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:

  mov eax,6
  mov ebx,4
  add eax,ebx
  mov [buf1],eax
  mov eax,buf1
  call sprintLF

  call quit

```

Рис 6.5 Изменение программы

В результате выполнения программы выводится символ с кодом 10, по таблице ASCII ему соответсвует перенос строки, поэтому программа переносит строку. (рис 6.6)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-1
```

Рис 6.6 Результат выполнения программы

Создаю новый файл lab6-2.asm. (рис 6.7)

```
asivanova@fedora:~/work/arch-pc/lab06$ touch lab6-2.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2.asm
```

Рис 6.7 Создание файла

Ввожу в новый файл текст программы из листинга 6.2. (рис 6.8)

Open

lab6-2.asm
~/work/arch-pc/lab06

```
%include 'in_out.asm'

SECTION .text
GLOBAL _start
  _start:

  mov eax,'6'
  mov ebx,'4'
  add eax,ebx
  call iprintLF

  call quit
```

Рис 6.8 Программа из листинга 6.2

В результате работы программы получаю число 106. В данном случае, как и в первом, команда add складывает коды символов ‘6’ и ‘4’, но функция iprintLF позволяет вывести число, а не символ, кодом которого является это число. (рис 6.9)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2.asm  lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2  lab6-2.asm  lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-2
106
```

Рис 6.9 Результат выполнения программы

Теперь снова изменяю символы на числа. (рис 6.10)

Open

lab6-2.asm
~/work/arch-pc/lab06

```
%include 'in_out.asm'

SECTION .text
GLOBAL _start
  _start:

  mov eax,6
  mov ebx,4
  add eax,ebx
  call iprintLF

  call quit
```

Рис 6.10 Замена символов на числа

В результате выполнения программы вполучаю число 10, сумму самих чисел 6 и 4. (рис 6.11)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2  lab6-2.asm  lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2  lab6-2.asm  lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-2
10
```

Рис 6.11 Результвт выполнения программы

Заменяю функцию iprintLF на iprint. (рис 6.12)

```
Open                                   lab6-2.asm
                                       ~/work/arch-pc/lab06
1  %include 'in_out.asm'
2
3  SECTION .text
4  GLOBAL _start
5    _start:
6
7    mov eax,6
8    mov ebx,4
9    add eax,ebx
10   call iprint
11
12   call quit
```

Рис 6.12 Замена подпрограммы

В итоге, как и с sprint вывод происходит на той же строке, без переноса (рис 6.13)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-2
10asivanova@fedora:~/work/arch-pc/lab06$ gedit lab6-2.asm
```

Рис 6.13 Результат работы измененной программы

6.3.2. Выполнение арифметических операций в NASM

Создаю новый файл. (рис 6.14)

```
asivanova@fedora:~/work/arch-pc/lab06$ touch lab6-3.asm
asivanova@fedora:~/work/arch-pc/lab06$ ls
in_out.asm  lab6-1  lab6-1.asm  lab6-1.o  lab6-2  lab6-2.asm  lab6-2.o  lab6-3.asm
```

Рис 6.14 Создание файла lab6-3.asm

Изучаю текст программы из листинга 6.3 и ввожу в lab6-3.asm. (рис 6.15)

```
Open                                  *lab6-3.asm
                                      ~/work/arch-pc/lab06
%include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
  _start:

  mov eax,5
  mov ebx,2
  mul ebx
  add eax,3
  xor edx, edx
  mov ebx,3
  div ebx

  mov edi,eax

  mov eax,div
  call sprint
  mov eax,edi
  call iprintLF

  mov eax,rem
  call sprint
  mov eax,edx
  call iprintLF

  call quit
```

Рис 6.15 Текст программы lab6-3

Создаю исполняемый файл и проверяю его работу. (рис 6.16)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
```

6.16 Проверка работы программы

Далее я изменяю текст программы для вычисления выражения $f(x) = (4 * 6 + 2)/5$. (рис 6.17)

```
%include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
  _start:

  mov eax,4
  mov ebx,6
  mul ebx
  add eax,2
  xor edx, edx
  mov ebx,5
  div ebx

  mov edi,eax

  mov eax,div
  call sprint
  mov eax,edi
  call iprintLF

  mov eax,rem
  call sprint
  mov eax,edx
  call iprintLF

  call quit
```

Рис 6.17 Изменение программы для вычисления другого выражения

Создаю исполняемый файл и проверяю его работу. (рис 6.18)

```
asivanova@fedora:~/work/arch-pc/lab06$ gedit lab6-3.asm
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
```

Рис 6.18 Проверка работы моей программы

Далее в новый файл я переписываю программу вычисления варианта задания по номеру студенческого билета, работающую по следующему алгоритму:

• вывести запрос на введение № студенческого билета

• вычислить номер варианта по формуле: $(Sn \bmod 20) + 1$, где Sn – номер студенческого билета

• вывести на экран номер варианта. (рис 6.19)

```
Open                                variant.asm
                                    ~/work/arch-pc/lab06
%include 'in_out.asm'

SECTION .data
 msg: DB 'Введите № студенческого билета: ',0
 rem: DB 'Ваш вариант: ',0

SECTION .bss
 x: RESB 80

SECTION .text
GLOBAL _start
  _start:

  mov eax,msg
  call sprintLF

  mov ecx,x
  mov edx,80
  call sread

  mov eax,x
  call atoi

  xor edx,edx
  mov ebx,20
  div ebx
  inc edx

  mov eax,rem
  call sprint
  mov eax,edx
  call iprintLF

  call quit
```

Рис 6.19 Программа, вычисляющая номер варианта

В результате программы я получаю 8 вариант. (рис 6.20)

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf variant.asm
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
asivanova@fedora:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032252467
Ваш вариант: 8
```

6.20 Результат работы программы

1. Какие строки листинга 6.4 отвечают за вывод на экран сообщения ‘Ваш вариант:’?

```
29  mov eax,rem
30  call sprint
```

2. Для чего используется следующие инструкции?

```
mov  ecx, x
mov  edx, 80
call sread
```

Инструкция mov ecx, x используется для записи адреса переменной x в регистр ecx, инструкция mov edx,80 записывает длину вводимого сообщения в ebx, а call sread – вызов подпрограммы, которая обеспечивает ввод сообщения.

3. Для чего используется инструкция “call atoi”?

Данная функция преобразовывает ascii-код символа в целое число и записывает результат в регистр eax.

4. Какие строки листинга 6.4 отвечают за вычисления варианта?

```
24  xor edx,edx
25  mov ebx,20
26  div ebx
27  inc edx
```

5. В какой регистр записывается остаток от деления при выполнении инструкции “div ebx”?

Остаток от деления записывается в регистр edx.

6. Для чего используется инструкция “inc edx”?

Для увеличения значения регистра edx на 1.

7. Какие строки листинга 6.4 отвечают за вывод на экран результата вычислений?

```
31  mov eax,edx
32  call iprintLF
```

Задание для самостоятельной работы

Пишу программу для вычисляения выражения в соответствии с моим вариантом. (рис 6.19) (рис 6.20) Программа работает корректно. (рис 6.21)

Open | lab6-4.asm ~/work/arch-pc/lab06 | Save

```
%include 'in_out.asm'

SECTION .data
 exp: DB '(11+x)*2-6',0
 msg: DB 'Введите значение x: ',0
 div: DB 'Результат: ',0

SECTION .bss
 x: RESB 80

SECTION .text
GLOBAL _start
  _start:

  mov eax,exp
  call sprintLF

  mov eax,msg
  call sprint

  mov ecx,x
  mov edx,80
  call sread

  mov eax,x
  call atoi

  add eax,11
  mov ebx,2
  mul ebx
  sub eax,6

  mov edi,eax

  mov eax,div
  call sprint
  mov eax,edi
  call iprintLF

  call quit
```

```
asivanova@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-4.asm
asivanova@fedora:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-4 lab6-4.o
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-4
(11+x)*2-6
Введите значение x: 1
Результат: 18
asivanova@fedora:~/work/arch-pc/lab06$ ./lab6-4
(11+x)*2-6
Введите значение x: 9
Результат: 34
```

Выводы

В процессе выполнения лабораторной работы я освоила арифметические инструкции языка ассемблера NASM.